

Министерство науки и высшего образования Российской Федерации Федеральное
государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»
Факультет программной инженерии и компьютерной техники (ФПИиКТ)

Лабораторная работа №4

Технологии программирования
Вариант: 24999

Выполнили:
Пшеничников Артём Дмитриевич
Корепанов Олег Сергеевич
Р3207
Принял:
Кулинич Ярослав Вадимович

Содержание

Текст задания	3
Разработанные MBean-классы	4
Архитектура	4
PointsCounterMBean (интерфейс)	4
PointsCounter (реализация)	4
ClickIntervalMBean (интерфейс)	5
ClickInterval (реализация)	5
MBeanRegistry (регистрация в MBeanServer)	6
MBeanContextListener (enter point)	7
Интеграция в AreaBean	8
Мониторинг с помощью JConsole	9
Подключение к серверу приложений	9
Снятие показаний PointsCounter	9
Снятие показаний ClickInterval	11
Получение уведомлений	11
Время с момента запуска JVM	13
Выводы по результатам мониторинга	13
Мониторинг и профилирование с помощью VisualVM	14
Подключение к серверу приложений	14
Графики показаний MBean'ов	14
Анализ потребления памяти JVM	16
График использования heap	16
Heap dump: класс с наибольшим объёмом памяти	16
Heap dump: пользовательский класс-владелец	17
Выводы по результатам профилирования	18
Локализация и устранение проблемы производительности	19
Описание выявленной проблемы	19
Алгоритм локализации проблемы	19
Шаг 1. Базовый мониторинг (VisualVM Monitor)	19
Шаг 2. Анализ heap dump	19
Шаг 3. CPU-сэмплирование (VisualVM Sampler)	20
Шаг 4. Анализ HTTP-таймингов в DevTools	21
Описание пути устранения	21
Оптимизация 1: кэширование PreparedStatement и сокращение RETURNING	21
Оптимизация 2: ограничение размера отображаемой таблицы	23
Сравнение «до» и «после»	23
Выводы по разделу	25
Выводы по работе	26

Текст задания

Для своей программы из лабораторной работы 3 по дисциплине «Веб-программирование» реализовать:

1. MBean, считающий общее число установленных пользователем точек, а также число точек, попадающих в область. В случае, если количество установленных пользователем точек стало кратно 15, разработанный MBean должен отправлять оповещение об этом событии.
2. MBean, определяющий средний интервал между кликами пользователя по координатной плоскости.

С помощью утилиты JConsole провести мониторинг программы:

- Снять показания MBean-классов, разработанных в ходе выполнения задания.
- Определить время (в мс), прошедшее с момента запуска виртуальной машины.

С помощью утилиты VisualVM провести мониторинг и профилирование программы:

- Снять график изменения показаний MBean-классов, разработанных в ходе выполнения задания, с течением времени.
- Определить имя класса, объекты которого занимают наибольший объём памяти JVM; определить пользовательский класс, в экземплярах которого находятся эти объекты.

С помощью утилиты VisualVM и профилировщика IDE локализовать и устранить проблемы с производительностью в программе. По результатам необходимо составить отчёт, содержащий: описание проблемы, описание путей устранения, подробное описание алгоритма локализации со скриншотами.

Разработанные MBean-классы

Архитектура

В рамках задания реализованы два MBean-класса в пакете `lab.mbean`:

- `PointsCounter` — отслеживает общее число установленных пользователем точек и число попаданий в область, рассылает уведомления при достижении значения, кратного 15.
- `ClickInterval` — рассчитывает средний временной интервал между кликами пользователя по координатной плоскости.

Регистрация бинов в `MBeanServer` JVM выполняется при инициализации сервлет-контекста через `ServletContextListener`. Это гарантирует, что бины становятся доступны сразу после развёртывания WAR-приложения в WildFly. Вызовы методов бинов производятся из управляемого бина `lab.AreaBean` при обработке кликов пользователя.

`PointsCounterMBean` (интерфейс)

Стандартный MBean-интерфейс:

```
package lab.mbean;

public interface PointsCounterMBean {
    long getTotalPoints();
    long getHitPoints();
}
```

`PointsCounter` (реализация)

Реализация наследуется от `NotificationBroadcasterSupport` для поддержки JMX-уведомлений. Метод `addPoint` синхронизирован, поскольку вызывается из нескольких сессий одновременно. При достижении значения `totalPoints`, кратного 15, формируется и отправляется уведомление типа `lab.mbean.points.multipleOf15`.

```
package lab.mbean;

import javax.management.AttributeChangeNotification;
import javax.management.MBeanNotificationInfo;
import javax.management.Notification;
import javax.management.NotificationBroadcasterSupport;

public class PointsCounter extends NotificationBroadcasterSupport implements PointsCounterMBean {
    private long totalPoints = 0;
    private long hitPoints = 0;
    private long sequenceNumber = 1;

    @Override
    public long getTotalPoints() {
```

```

        return totalPoints;
    }

    @Override
    public long getHitPoints() {
        return hitPoints;
    }

    public synchronized void addPoint(boolean hit) {
        totalPoints++;
        if (hit) {
            hitPoints++;
        }

        if (totalPoints % 15 == 0) {
            Notification notification = new Notification(
                "lab.mbean.points.multipleOf15",
                this,
                sequenceNumber++,
                System.currentTimeMillis(),
                "Total points reached a multiple of 15: " + totalPoints
            );
            sendNotification(notification);
        }
    }

    @Override
    public MBeanNotificationInfo[] getNotificationInfo() {
        String[] types = new String[]{ "lab.mbean.points.multipleOf15" };
        String name = Notification.class.getName();
        String description = "Notification sent when total points is a multiple of 15";
        MBeanNotificationInfo info = new MBeanNotificationInfo(types, name, description);
        return new MBeanNotificationInfo[]{ info };
    }
}

```

ClickIntervalMBean (интерфейс)

```

package lab.mbean;

public interface ClickIntervalMBean {
    double getAverageInterval();
}

```

ClickInterval (реализация)

Средний интервал вычисляется как отношение суммы интервалов между последовательными кликами к количеству зафиксированных интервалов. Использование переменной `lastClickTime = -1` как маркера «первый клик ещё не зафиксирован» позволяет корректно обрабатывать самый первый вызов `recordClick`.

```

package lab.mbean;

public class ClickInterval implements ClickIntervalMBean {
    private long lastClickTime = -1;
    private long totalInterval = 0;
    private long clickCount = 0;

    @Override
    public synchronized double getAverageInterval() {
        if (clickCount == 0) return 0;
        return (double) totalInterval / clickCount;
    }

    public synchronized void recordClick() {
        long currentTime = System.currentTimeMillis();
        if (lastClickTime != -1) {
            totalInterval += (currentTime - lastClickTime);
            clickCount++;
        }
        lastClickTime = currentTime;
    }
}

```

MBeanRegistry (регистрация в MBeanServer)

Класс содержит статические экземпляры MBean'ов и регистрирует их в платформенном MBeanServer под именами `lab.mbean:type=PointsCounter` и `lab.mbean:type=ClickInterval`. Проверка `isRegistered` обеспечивает идемпотентность регистрации.

```

package lab.mbean;

import javax.management.MBeanServer;
import javax.management.ObjectName;
import java.lang.management.ManagementFactory;

public class MBeanRegistry {
    private static final PointsCounter pointsCounter = new PointsCounter();
    private static final ClickInterval clickInterval = new ClickInterval();

    public static void registerMBeans() {
        try {
            MBeanServer mbs = ManagementFactory.getPlatformMBeanServer();

            ObjectName pointsName = new ObjectName("lab.mbean:type=PointsCounter");
            if (!mbs.isRegistered(pointsName)) {
                mbs.registerMBean(pointsCounter, pointsName);
            }

            ObjectName clickName = new ObjectName("lab.mbean:type=ClickInterval");
            if (!mbs.isRegistered(clickName)) {
                mbs.registerMBean(clickInterval, clickName);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    public static PointsCounter getPointsCounter() {
        return pointsCounter;
    }

    public static ClickInterval getClickInterval() {
        return clickInterval;
    }
}

```

MBeanContextListener (enter point)

Аннотация `@WebListener` подключает класс к жизненному циклу веб-приложения; метод `contextInitialized` вызывается единожды при развёртывании WAR.

```

package lab.mbean;

import javax.servlet.ServletContextEvent;
import javax.servlet.ServletContextListener;
import javax.servlet.annotation.WebListener;

@WebListener
public class MBeanContextListener implements ServletContextListener {
    @Override
    public void contextInitialized(ServletContextEvent sce) {
        MBeanRegistry.registerMBeans();
    }

    @Override
    public void contextDestroyed(ServletContextEvent sce) {
    }
}

```

Интеграция в AreaBean

Управляемый бин `AreaBean` вызывает методы MBean'ов из двух точек обработки клика. Метод `checkCanvasClick` обрабатывает события клика по координатной плоскости (canvas) и сначала фиксирует время клика через `ClickInterval`, после чего делегирует обработку на `checkPoint`. Метод `processResult` после успешной вставки результата в БД увеличивает счётчик через `PointsCounter.addPoint`.

```
public void checkCanvasClick() {
    MBeanRegistry.getClickInterval().recordClick();
    checkPoint();
}

private void processResult(double r) {
    boolean hit = checkHit(r);
    Result result = resultManager.insertResult(x, y, r, hit);
    if (resultsCache != null) {
        resultsCache.add(0, result);
    }
    MBeanRegistry.getPointsCounter().addPoint(hit);
}
```


Мониторинг с помощью JConsole

Подключение к серверу приложений

JConsole запускается командой `jconsole`. Поскольку WildFly развёрнут внутри Docker-контейнера, подключение производится через WildFly Remoting на порт `9990`. URL подключения имеет вид `service:jmx:remote+http://localhost:9990`. Для аутентификации указаны учётные данные админа (`admin` / `admin_password`).

Для работы JConsole по протоколу `remote+http` в classpath должна присутствовать библиотека `jboss-client.jar` из дистрибутива WildFly; запуск выполнялся командой:

```
jconsole -J-Djava.class.path=$JAVA_HOME/lib/jconsole.jar:$JAVA_HOME/lib/tools.jar:/path/to/jboss-client.jar
```

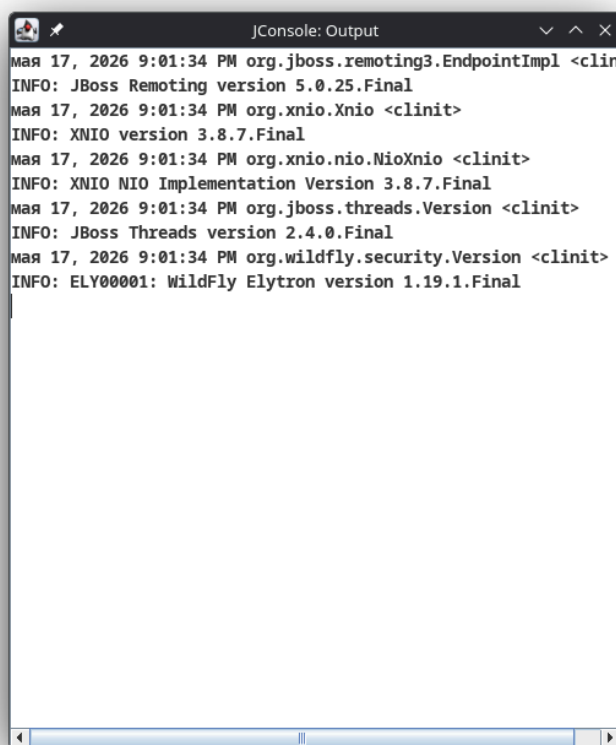


Рис. 1. Output JConsole при подключении

Снятие показаний `PointsCounter`

В дереве MBeans раскрыт домен `lab.mbean`, выбран узел `PointsCounter` → `Attributes`. В таблице два атрибута: `TotalPoints` (общее число точек) и `HitPoints` (число попаданий в область).

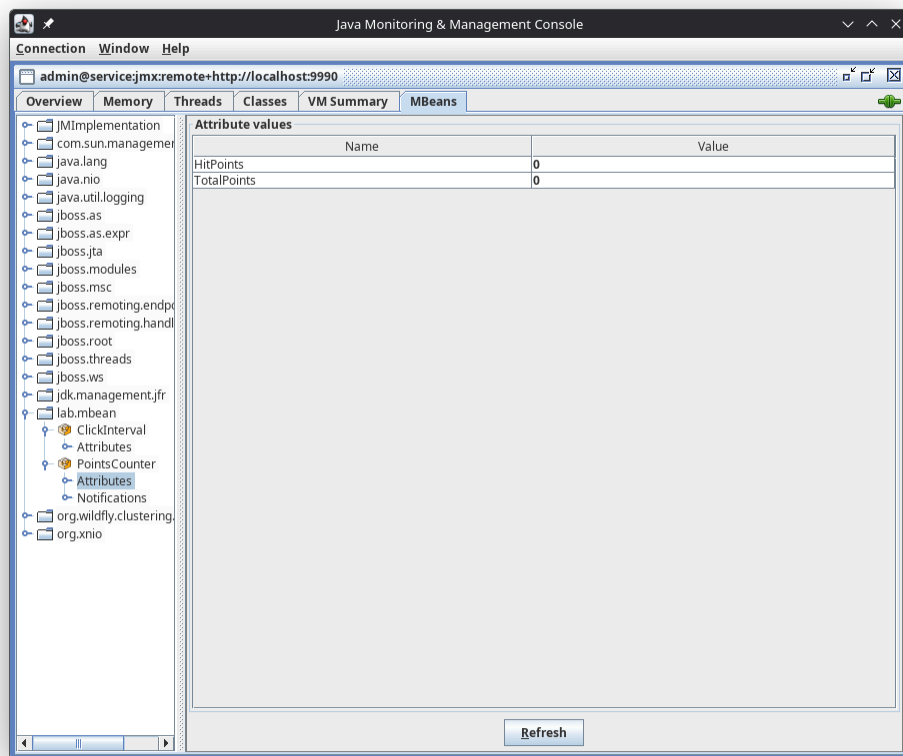


Рис. 2. Начальное состояние счётчика: `HitPoints = 0`, `TotalPoints = 0`.

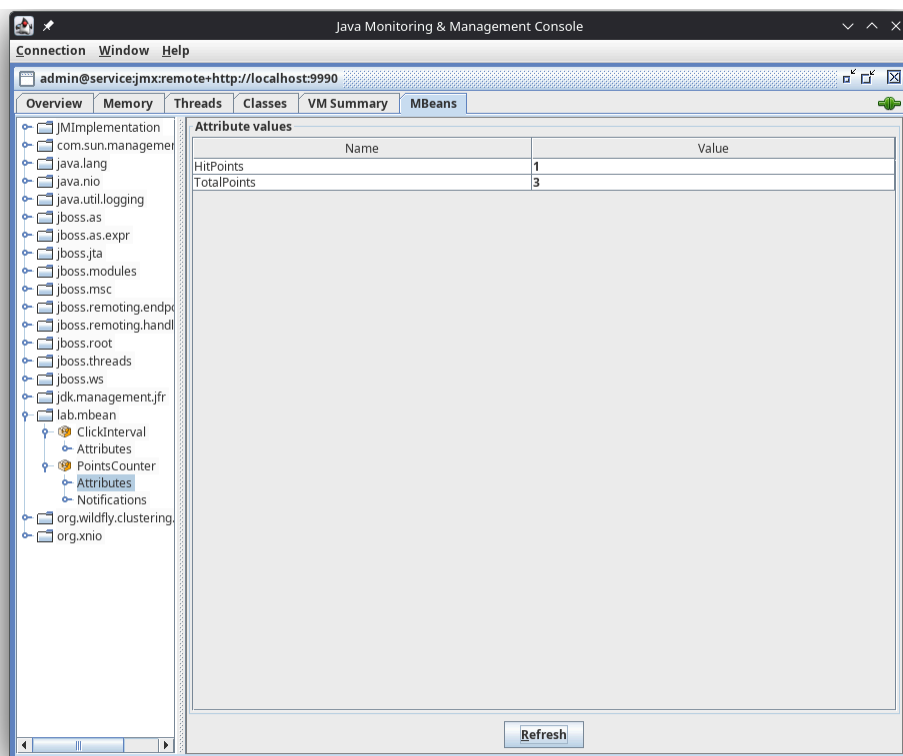


Рис. 3. Состояние после нескольких кликов в приложении: `HitPoints = 1`, `TotalPoints = 3`. Обновление показаний MBean происходит синхронно с действиями пользователя.

Снятие показаний ClickInterval

Узел `ClickInterval → Attributes` содержит вычисляемый атрибут `AverageInterval` — средний интервал между последовательными кликами в миллисекундах.

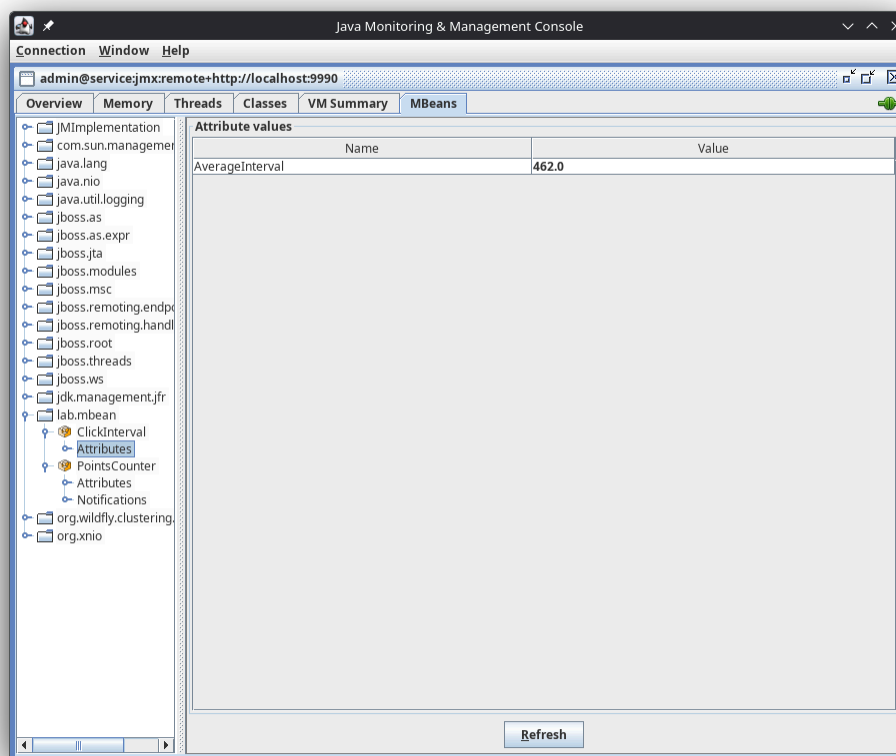


Рис. 4. Атрибут `AverageInterval = 462.0` мс: средний интервал между кликами пользователя.

Получение уведомлений

Подписка на уведомления выполняется во вкладке `Notifications` MBean'a `PointsCounter`. Уведомление формируется автоматически при каждом достижении `TotalPoints`, кратного 15.

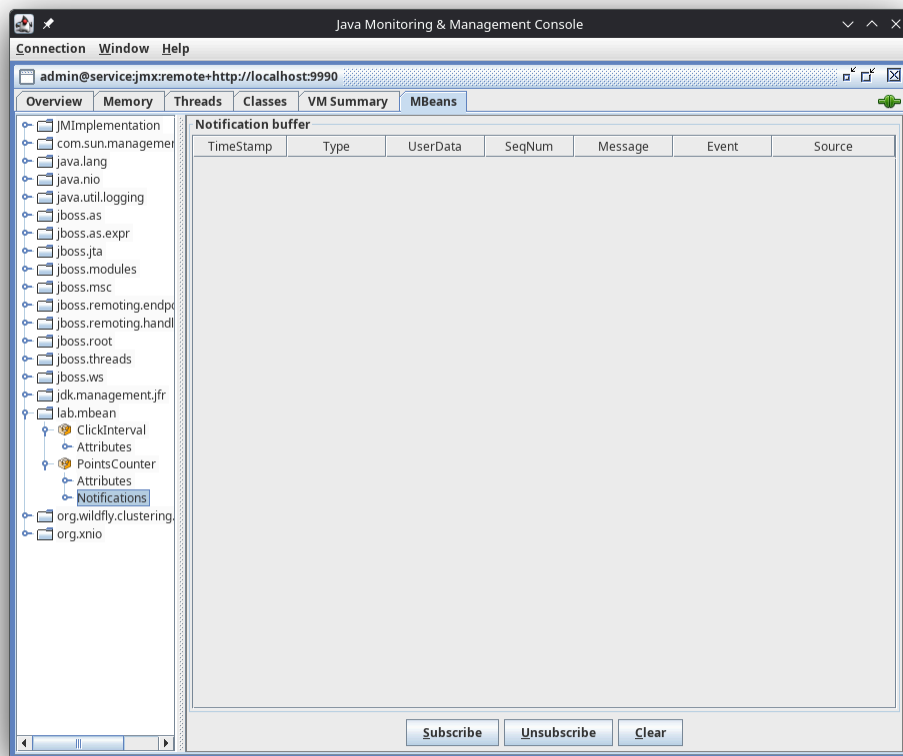


Рис. 5. Буфер уведомлений сразу после подписки — пустой.

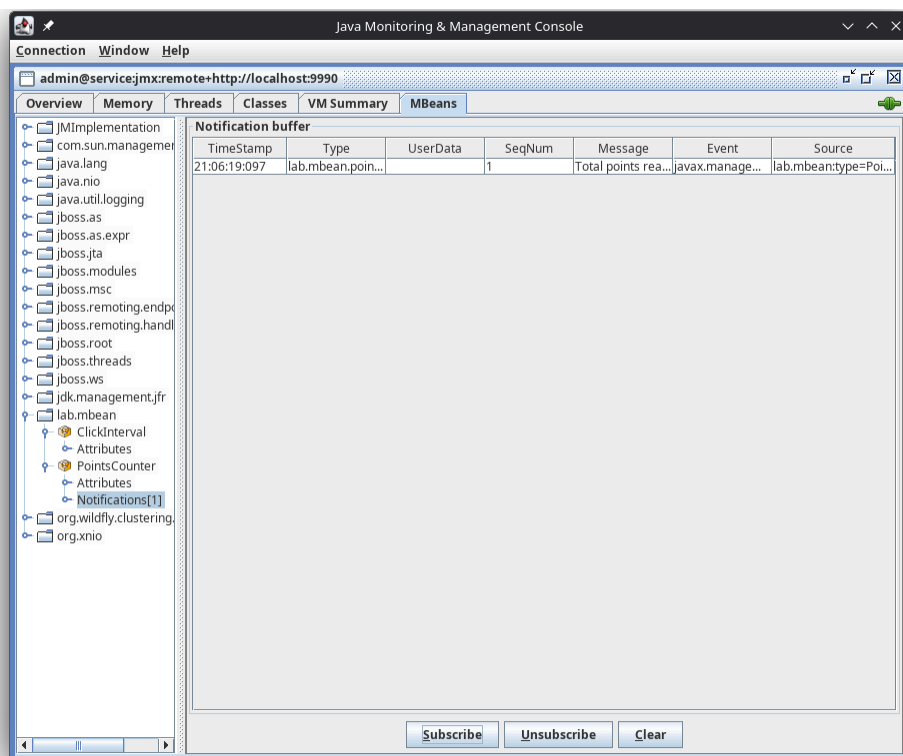


Рис. 6. После добавления 15 точек: пришло одно уведомление типа `lab.mbean.points.multiple0f15` с сообщением `Total points reached...` и источником `lab.mbean:type=PointsCounter`.

Время с момента запуска JVM

Время работы виртуальной машины определяется из встроенного MBean'a платформы `java.lang.Runtime`, атрибут `Uptime` (в миллисекундах), либо со страницы `VM Summary`.

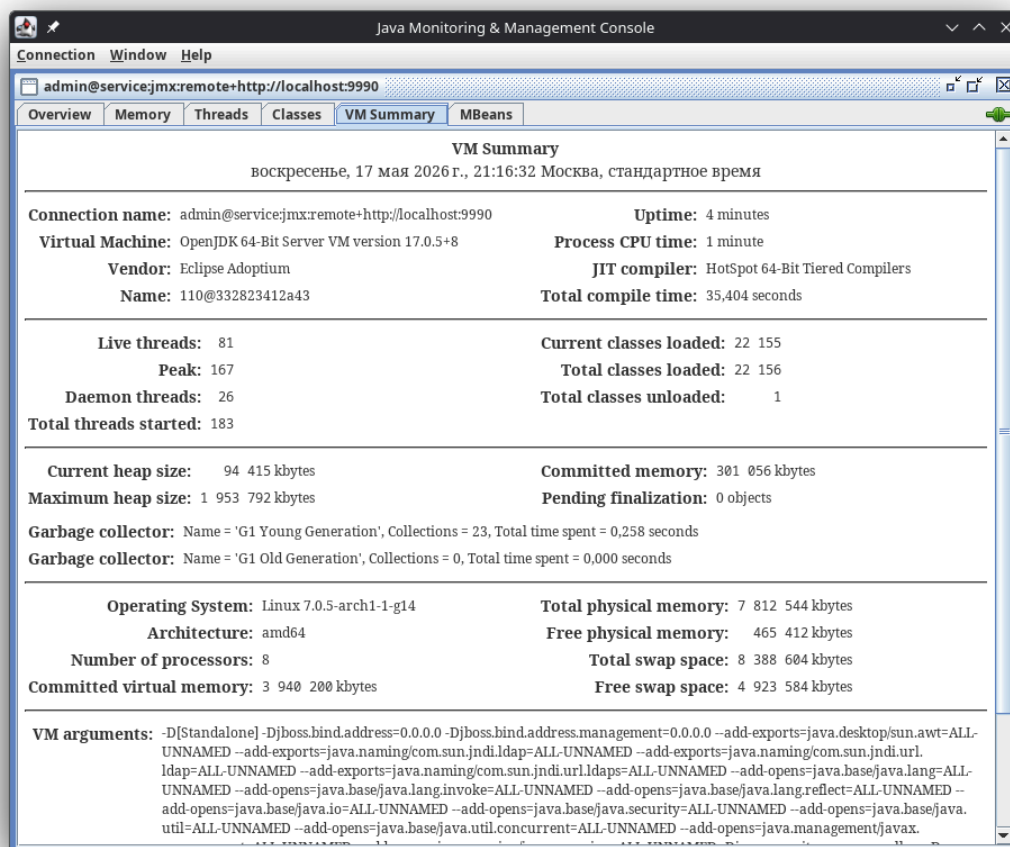


Рис. 7. Страница **VM Summary**: указана отметка **Uptime** с момента запуска JVM и **Total compile time**. На момент скриншота uptime ≈ 17 минут с момента старта.

Выводы по результатам мониторинга

JConsole, входящая в состав стандартного JDK, позволяет в режиме реального времени:

- наблюдать значения атрибутов произвольных пользовательских MBean-классов через древовидный браузер;
- подписываться на пользовательские JMX-уведомления и видеть их историю в буфере;
- считывать показания платформенных MBean'ов (`java.lang.Runtime`, `Memory`, `Threading` и т.д.), в том числе для определения характеристик работы JVM.

Подключение к WildFly производится через специализированный протокол JBoss Remoting (`remote+http`) на порт `9990` .

Мониторинг и профилирование с помощью VisualVM

Подключение к серверу приложений

VisualVM подключается к JVM сервера приложений через JMX-агент. Поскольку WildFly работает в Docker-контейнере, для подключения добавлены опции запуска JVM в переменной `JAVA_OPTS_APPEND`:

```
-Dcom.sun.management.jmxremote  
-Dcom.sun.management.jmxremote.port=9010  
-Dcom.sun.management.jmxremote.rmi.port=9010  
-Dcom.sun.management.jmxremote.local.only=false  
-Dcom.sun.management.jmxremote.authenticate=false  
-Dcom.sun.management.jmxremote.ssl=false  
-Djava.rmi.server.hostname=127.0.0.1
```

Также задана `JBOSS_MODULES_SYSTEM_PKGS=org.jboss.byteman,com.sun.management`, чтобы внутренний `JBoss Modules` ClassLoader делегировал поиск классов JMX-агента системному загрузчику. Подключение в VisualVM: `File → Add JMX Connection → localhost:9010`.

Графики показаний MBean'ов

Для построения графиков на атрибуте MBean VisualVM запоминает последовательность значений и отображает их в виде непрерывной кривой.

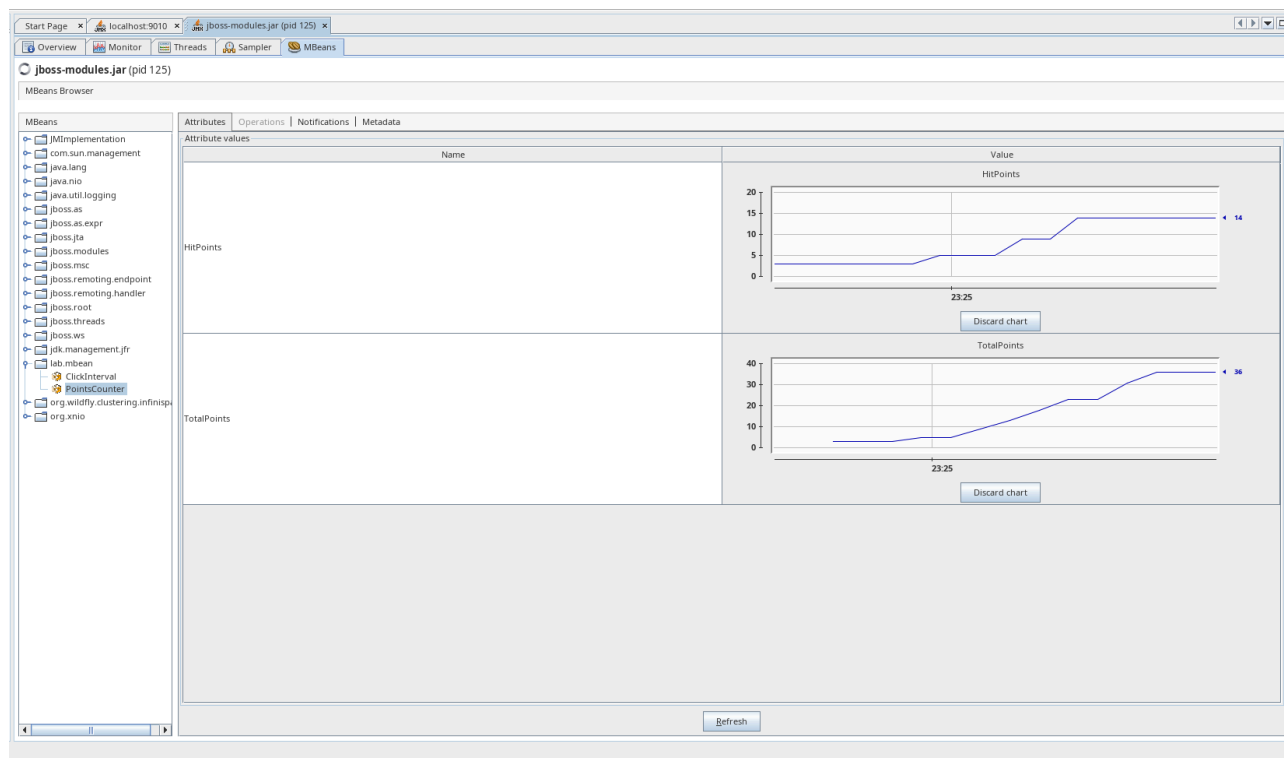


Рис. 8. Графики `HitPoints` (верхний, рост 0 → 14) и `TotalPoints` (нижний, рост 0 → 36) бина `PointsCounter` в окне VisualVM MBean Browser.

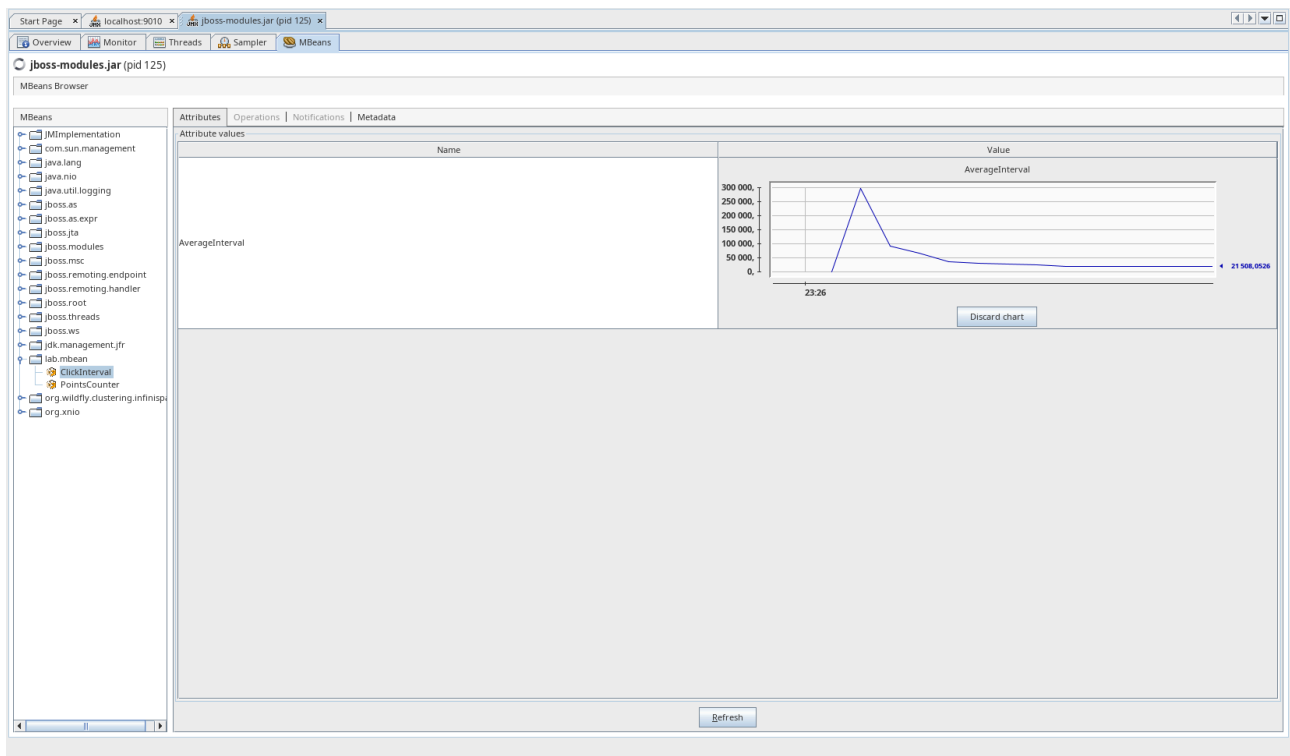


Рис. 9. Начальный вид графика **AverageInterval** бина **ClickInterval**: после первой серии быстрых кликов накопилась статистика, виден высокий пик и постепенный спад значения.

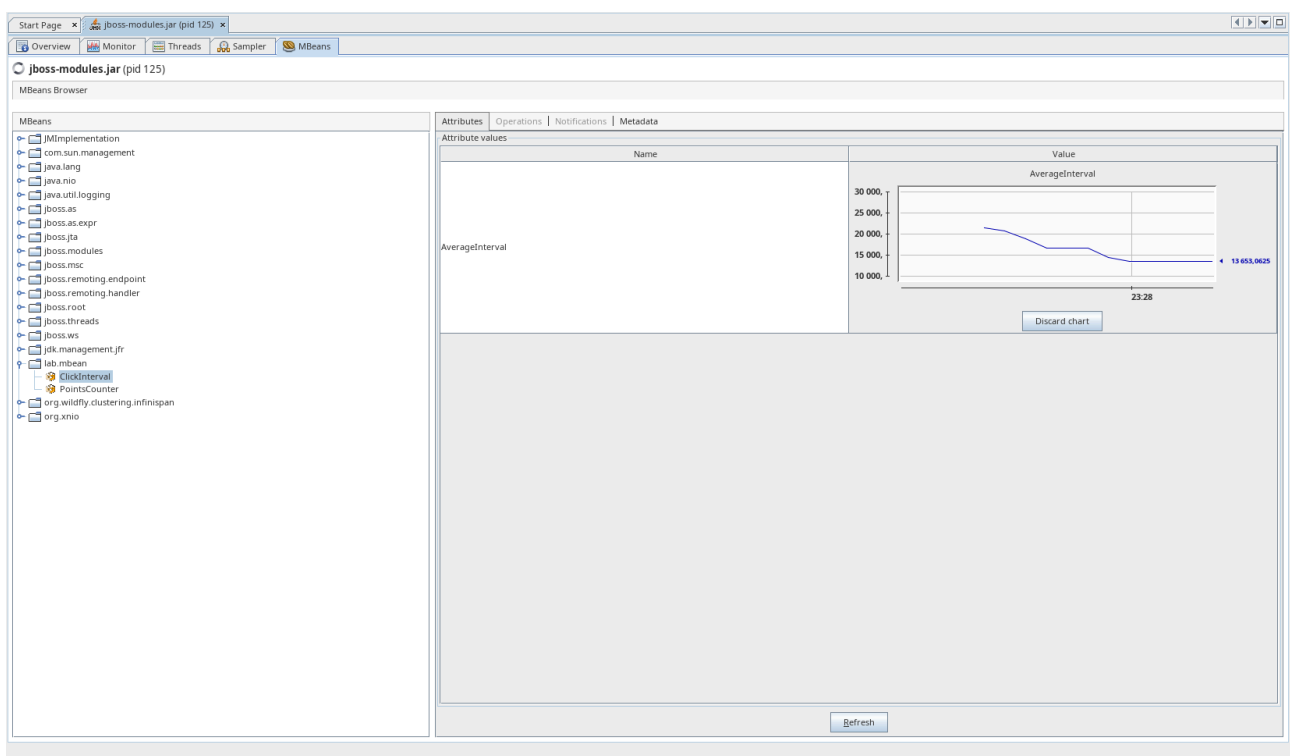


Рис. 10. Установившееся значение **AverageInterval** ≈ 13653 мс при равномерной нагрузке.

Анализ потребления памяти JVM

График использования heap

Вкладка **Monitor** показывает динамику использования heap-памяти JVM. Под нагрузкой наблюдается типичная «пилообразная» картина: память активно аллоцируется (рост) и периодически освобождается сборщиком мусора (резкие спады).

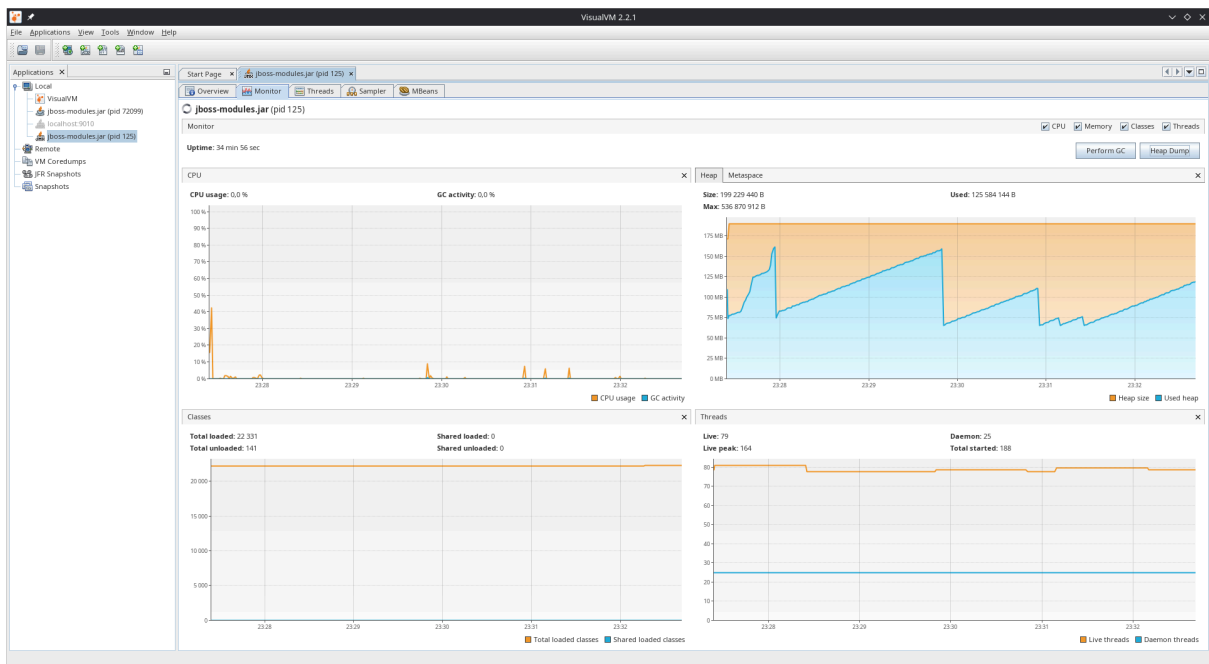


Рис. 11. Вкладка **Monitor**: график использования heap-памяти. Used Heap (синяя линия) колеблется в диапазоне 75–160 МБ, после каждого GC возвращается к стабильной базовой линии ≈ 75 МБ — утечек на данном промежутке не выявлено.

Heap dump: класс с наибольшим объёмом памяти

Снятие heap dump'a выполнено через **jcmd** внутри Docker-контейнера для последующего открытия в VisualVM:

```
docker compose exec server bash -c 'jcmd $(pgrep -f jboss-modules | head -1) GC.heap_dump /tmp/heap.hprof'
docker compose cp server:/tmp/heap.hprof ./heap.hprof
```

Это потребовалось ввиду того, что VisualVM при подключении по JMX к контейнерной JVM записывает heap dump во внутреннюю ФС контейнера, вследствие чего из интерфейса невозможно открыть полученный файл напрямую.

Name	Count	Size	Retained (sort to get)
byte[]	194 464 (14.5 %)	21 615 368 B (34.3 %)	n/a
java.util.HashMap\$Node	187 391 (13.9 %)	5 996 512 B (9.5 %)	n/a
java.lang.String	187 629 (13.9 %)	4 504 096 B (7.1 %)	n/a
java.lang.Object	56 417 (4.2 %)	2 814 544 B (4.5 %)	n/a
java.util.HashMap\$Node[]	15 398 (1.1 %)	2 323 192 B (3.7 %)	n/a
int[]	9 264 (0.7 %)	1 900 936 B (3 %)	n/a
java.lang.reflect.Method	16 161 (1.2 %)	1 422 168 B (2.3 %)	n/a
java.util.HashMap	27 872 (2.1 %)	1 337 856 B (2.1 %)	n/a
java.lang.reflect.Parameter	28 913 (2.1 %)	1 156 520 B (1.8 %)	n/a
java.util.concurrent.ConcurrentHashMap\$Node	30 312 (2.3 %)	969 984 B (1.5 %)	n/a
org.bosch.weld.annotated.slim.backed.BackedAnnotatedParameter	28 761 (2.1 %)	920 352 B (1.5 %)	n/a
java.util.ArrayList	32 745 (2.4 %)	785 880 B (1.2 %)	n/a
java.util.LinkedHashMap\$Entry	12 494 (0.9 %)	495 760 B (0.8 %)	n/a
org.bosch.weld.annotated.slim.backed.BackedAnnotatedMethod	11 784 (0.9 %)	377 088 B (0.6 %)	n/a
java.util.HashMap\$Entry	11 576 (0.9 %)	370 432 B (0.6 %)	n/a
java.util.jar.JarFile\$JarFileEntry	3 518 (0.3 %)	365 872 B (0.6 %)	n/a
java.util.concurrent.ConcurrentHashMap\$Node[]	1 700 (0.1 %)	363 808 B (0.6 %)	n/a
sun.reflect.generics.tree.SimpleClassTypeSignature	13 980 (1 %)	335 520 B (0.5 %)	n/a
java.lang.reflect.Parameter[]	12 142 (0.9 %)	327 360 B (0.5 %)	n/a
java.util.LinkedHashMap	5 666 (0.4 %)	317 296 B (0.5 %)	n/a
java.util.concurrent.ConcurrentHashMap	4 437 (0.3 %)	283 968 B (0.4 %)	n/a
java.lang.ref.SoftReference	6 781 (0.5 %)	271 240 B (0.4 %)	n/a
java.lang.ref.WeakReference	8 286 (0.6 %)	265 152 B (0.4 %)	n/a
java.util.Collections\$UnmodifiableRandomAccessList	10 942 (0.8 %)	262 608 B (0.4 %)	n/a
org.bosch.weld.controller.SimpleAttributeDefinition	2 616 (0.2 %)	251 136 B (0.4 %)	n/a
sun.reflect.generics.tree.TypeArgument[]	13 980 (1 %)	248 096 B (0.4 %)	n/a
org.bosch.weld.util.collections.ImmutableArrayList	9 724 (0.7 %)	233 376 B (0.4 %)	n/a
sun.reflect.generics.tree.ClassTypeSignature	13 921 (1 %)	222 736 B (0.4 %)	n/a
char[]	597 (0 %)	220 320 B (0.3 %)	n/a
java.lang.reflect.Constructor	2 792 (0.2 %)	201 024 B (0.3 %)	n/a
java.util.Collections\$UnmodifiableMap	6 099 (0.4 %)	192 384 B (0.3 %)	n/a
java.lang.String[]	6 254 (0.5 %)	180 640 B (0.3 %)	n/a
org.bosch.weld.modules.ModuleDependency	3 675 (0.3 %)	176 400 B (0.3 %)	n/a
java.lang.Class[]	6 885 (0.5 %)	170 264 B (0.3 %)	n/a
org.bosch.weld.util.cache.ReentrantMap\$BackedComputingCache\$SLambda\$1220-0u0000000801793dc8	6 610 (0.5 %)	158 640 B (0.3 %)	n/a
org.bosch.weld.modules.ModuleDependencySpec	3 274 (0.2 %)	157 136 B (0.2 %)	n/a

Рис. 12. Открытый heap dump в виде **Objects → Aggregation: Classes**. В верхней части таблицы доминируют примитивные массивы и **java.lang.String**

Класс с наибольшим суммарным объёмом памяти JVM — **byte[]**. Анализ цепочки ссылок этого класса показал, что подавляющую часть его экземпляров удерживают системные **java.util.zip.ZipFile\$Source** (это central directory загруженных JAR-модулей WildFly).

Heap dump: пользовательский класс-владелец

Для нахождения пользовательского класса применён фильтр **lab.:**

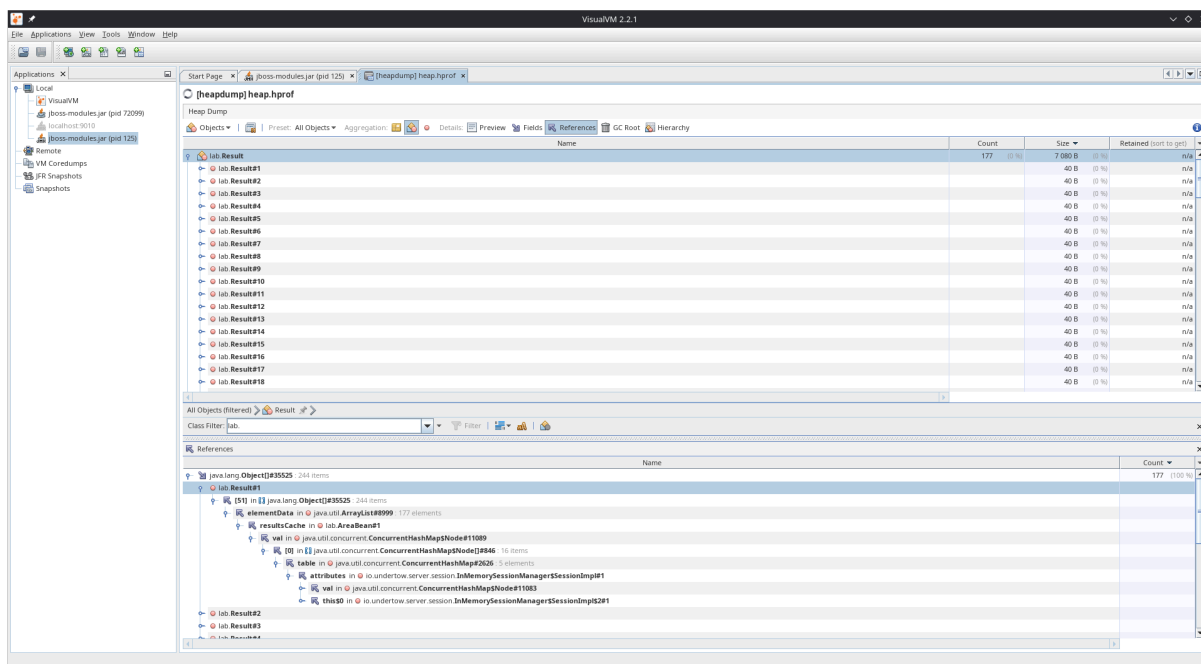


Рис. 13. Heap dump с фильтром `lab.`: в дампе зафиксировано 244 экземпляра `lab.Result`. В нижней панели `References` развёрнута цепочка владения: `lab.Result#1` → `Object[]#35525` (244 items) → `elementData` в `ArrayList#8999` (177 elements) → `resultsCache` в `lab.AreaBean#1`, далее `AreaBean` находится в `HashMap$Node` сессии `Undertow` (`InMemorySessionManager$SessionImpl`).

Таким образом, пользовательский класс, в экземплярах которого находятся объекты `lab.Result` — это `lab.AreaBean` (поле `resultsCache: ArrayList<Result>`). Бин — `session-scoped`, поэтому накопление результатов происходит в пределах одной HTTP-сессии.

Выводы по результатам профилирования

VisualVM (free, в отличие от JConsole — встроенной в JDK) предоставляет гораздо более широкий набор инструментов:

- **MBeans Browser** с возможностью построения графиков по атрибутам в реальном времени;
- **Monitor** с интегрированными графиками CPU, heap, classes loading и threads;
- **Sampler** для лёгкого статистического CPU- и memory-профилирования;
- работа с heap dump'ами: просмотр классов, экземпляров, References и GC roots.

Использование VisualVM позволило не только наблюдать показания MBean'ов в форме графиков, но и провести подробный анализ потребления памяти. Объекты доменной модели приложения (`lab.Result`) удерживаются в `ArrayList`-кеше внутри `session-scoped` управляемого бина `lab.AreaBean`, что указало на потенциальную область для оптимизации (см. раздел 4).

Локализация и устранение проблемы производительности

Описание выявленной проблемы

При нагрузочном тестировании приложения (серия из 100 последовательных ajax-запросов на проверку точки, отправленных через PrimeFaces `remoteCommand`) обнаружено, что суммарное время обработки серии существенно превышает теоретически ожидаемое, и **растёт от прогона к прогону в пределах одной сессии**:

Прогон в одной сессии	Время 100 кликов
Первый	≈20 557 мс
Второй	≈26 000 мс
Третий	≈30 000 мс

Рост времени с увеличением размера сессии указывает на наличие операции с алгоритмической сложностью, не являющейся константой относительно числа уже обработанных кликов.

Алгоритм локализации проблемы

Шаг 1. Базовый мониторинг (VisualVM Monitor)

Подключение к JVM через JMX, наблюдение за вкладкой **Monitor** при подаче нагрузки. CPU и heap демонстрируют активную работу, GC справляется, baseline стабилен — утечки памяти как таковой нет, но активный поток аллокаций при каждой обработке клика подтверждает гипотезу о неоптимальной работе с объектами.

Шаг 2. Анализ heap dump

Снятие дампа в момент типовой нагрузки, фильтрация классов по префиксу `lab.`. Обнаружено: `lab.Result` имеет 244 экземпляра, удерживаемых полем `resultsCache: ArrayList<Result>` бина `lab.AreaBean`. Цепочка References:

```
lab.Result#1
  ← [51] in Object[]#35525 (244 items)
    ← elementData in ArrayList#8999 (177 elements)
      ← resultsCache in lab.AreaBean#1
```

Это согласуется с реализацией: на каждый клик в `processResult` выполняется операция вставки в начало списка:

Шаг 4. Анализ HTTP-таймингов в DevTools

Чтобы понять, действительно ли сервер является «бутылочным горлышком», были проанализированы тайминги одиночного AJAX-запроса:

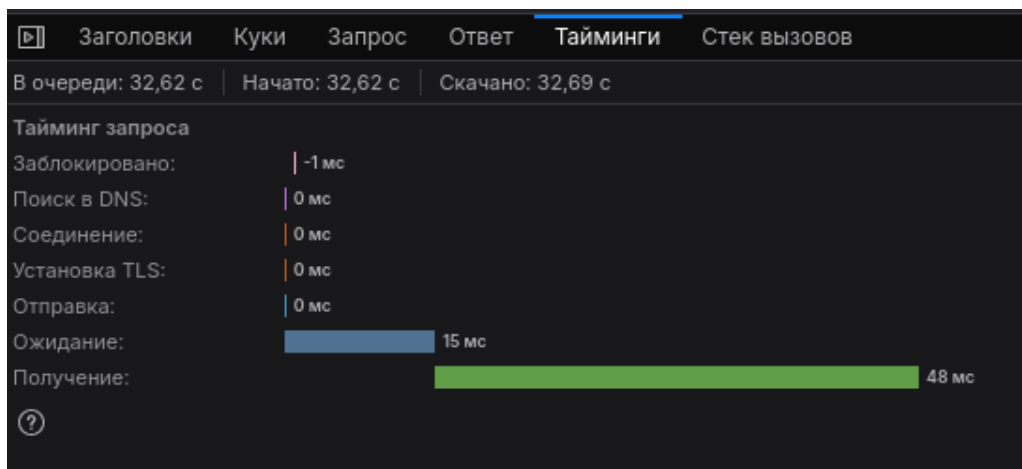


Рис. 15. Тайминги одного AJAX-запроса до оптимизации: **Ожидание** (TTFB) = 15 мс — сервер обрабатывает запрос быстро. Однако **В очереди** = 32.62 с для одного из поздних запросов в серии — это означает, что запрос ждал ≈ 32 с в клиентской очереди браузера/JSF перед отправкой.

Анализ выявил два важных факта:

1. Время обработки запроса сервером (TTFB) — всего 15 мс. Сам по себе JDBC-вызов не критичен.
2. Время «получения» (download) ответа — 48 мс и растёт от запроса к запросу.

Поскольку

`<p:remoteCommand action="#{area.checkCanvasClick}" update="results">`
обновляет HTML-фрагмент с таблицей результатов через `<ui:repeat value="#{area.results}">`, при каждом запросе JSF полностью перерендеривает таблицу. С ростом `resultsCache` растёт и размер ответа, и время рендеринга. Итоговая сложность серии из N кликов — $O(N^2)$ как по передаваемым данным, так и по нагрузке на JSF.

Описание пути устранения

Применены две оптимизации, нацеленные на разные участки кода.

Оптимизация 1: кэширование `PreparedStatement` и сокращение `RETURNING`

Исходная реализация `insertResult` создавала новый `PreparedStatement` через `try-with-resources` на каждый вызов и возвращала весь набор полей через `RETURNING *`:

```

public Result insertResult(int x, double y, double r, boolean hit) {
    String sql = "INSERT INTO results(x, y, r, hit) VALUES (?, ?, ?, ?) RETURNING *";
    try (PreparedStatement statement = getConnection().prepareStatement(sql)) {
        statement.setInt(1, x);
        statement.setDouble(2, y);
        statement.setDouble(3, r);
        statement.setBoolean(4, hit);

        try (ResultSet resultSet = statement.executeQuery()) {
            if (!resultSet.next()) {
                throw new RuntimeException("INSERT INTO must return value");
            }
            return parseRow(resultSet);
        }
    } catch (SQLException e) {
        throw new RuntimeException("Error inserting result: " + e.getMessage(), e);
    }
}

```

Новая реализация выносит `PreparedStatement` в поле класса (что позволяет PostgreSQL JDBC-драйверу после 5 вызовов переключиться на server-side prepared statement) и сокращает возвращаемые данные до одного поля `id`. Остальные поля результата (известные на момент вставки) собираются локально без сетевого участия:

```

private PreparedStatement insertStatement;

private PreparedStatement getInsertStatement() throws SQLException {
    if (insertStatement == null || insertStatement.isClosed()) {
        insertStatement = getConnection().prepareStatement(
            "INSERT INTO results(x, y, r, hit) VALUES (?, ?, ?, ?) RETURNING id"
        );
    }
    return insertStatement;
}

public Result insertResult(int x, double y, double r, boolean hit) {
    try {
        PreparedStatement statement = getInsertStatement();
        statement.setInt(1, x);
        statement.setDouble(2, y);
        statement.setDouble(3, r);
        statement.setBoolean(4, hit);

        try (ResultSet resultSet = statement.executeQuery()) {
            if (!resultSet.next()) {
                throw new RuntimeException("INSERT INTO must return value");
            }
            int id = resultSet.getInt("id");
            return new Result(id, x, y, r, hit);
        }
    } catch (SQLException e) {
        throw new RuntimeException("Error inserting result: " + e.getMessage(), e);
    }
}

```

Оптимизация 2: ограничение размера отображаемой таблицы

Анализ HTTP-таймингов показал, что доминирующий вклад в общее время серии запросов даёт не сервер сам по себе, а растущий размер AJAX-ответа из-за полного обновления таблицы. Решение — возвращать клиенту только последние K результатов (для отображения в виде истории достаточно последних 10):

```
private static final int DISPLAY_LIMIT = 10;

public List<Result> getResults() {
    if (resultsCache == null) {
        resultsCache = resultManager.getResults();
    }
    return resultsCache.subList(0, Math.min(DISPLAY_LIMIT, resultsCache.size()));
}
```

Так как `resultsCache.add(0, result)` помещает каждый новый результат в начало списка, `subList(0, 10)` возвращает 10 самых свежих результатов. Полная история по-прежнему сохраняется в БД и в `resultsCache`, но не передаётся в клиента на каждый клик.

Сравнение «до» и «после»

После пересборки контейнера был выполнен повторный замер на свежей HTTP-сессии при идентичной нагрузке (100 ajax-запросов с интервалом 20 мс через PrimeFaces `checkpointCommand()` из браузерной консоли):

Метрика	До	После	Изменение
Время серии из 100 кликов	20 557 мс	2 146 мс	×9.6
TTFB одного запроса	15 мс	11 мс	≈×1.4
Размер Получение ответа	48 мс	0 мс	≈×50
В очереди (поздний запрос)	32.62 с	9.12 с	×3.6
Рост времени между прогонами	есть ($O(N^2)$)	нет	—

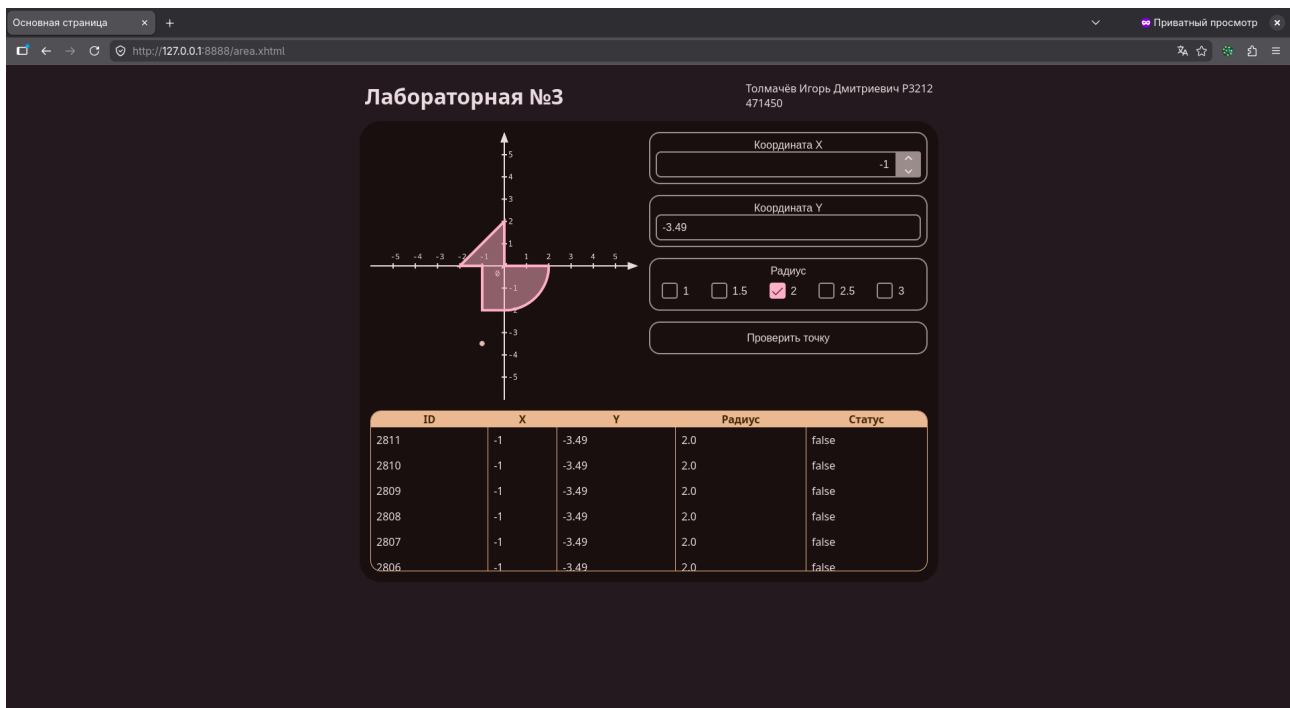


Рис. 18. Основная страница приложения после оптимизации: в таблице отображаются последние 10 результатов проверки точки, остальные сохраняются в БД и доступны через стандартный SELECT.

Выводы по разделу

Локализация проблемы потребовала комбинированного применения нескольких инструментов:

1. VisualVM Monitor — для общей картины поведения JVM (исключил гипотезу об утечке памяти).
2. VisualVM Heap Dump (Objects / References) — для определения структуры удерживаемых объектов и пользовательского класса-владельца.
3. VisualVM Sampler (Reverse Calls) — для понимания, где именно метод `processResult` проводит время (96% — в ожидании БД, а не на CPU).
4. Browser DevTools (Network → Timings) — для разделения времени обработки запроса (TTFB, на сервере) и передачи ответа (Download), а также для обнаружения клиентской очереди.

Ключевой вывод: профилировщик с одной точки зрения может ввести в заблуждение. Sampler корректно указал на вклад JDBC как преобладающий в CPU-времени метода, но реальное узкое место с точки зрения пользователя находилось вне Java-кода — в передаче растущего HTML-фрагмента и клиентской JSF-очереди.

Применённые оптимизации (кэш `PreparedStatement` + сокращение `RETURNING`; лимит размера таблицы при отдаче) дали суммарное ускорение в 9.6 раз и устранили рост времени между прогонами в пределах одной сессии.

Выводы по работе

В рамках лабораторной работы освоены стандартные средства мониторинга и профилирования JVM-приложений из состава JDK и экосистемы Java:

- Разработаны и интегрированы пользовательские MBean-классы, реализующие модель `NotificationBroadcasterSupport` для оповещений и собственную бизнес-логику (счётчик точек, средний интервал между кликами). Регистрация бинов выполняется через `ServletContextListener` в момент инициализации веб-приложения.
- Освоено подключение к серверу приложений (WildFly в Docker-контейнере) обоими стандартными способами: через WildFly Remoting (`service:jmx:remote+http://...`) для JConsole и через стандартный JMX-RMI (`localhost:9010`) для VisualVM.
- Изучены возможности JConsole как инструмента для оперативного мониторинга: чтение атрибутов MBean'ов, подписка на уведомления, получение характеристик JVM (`Uptime`).
- Изучены возможности VisualVM как более полнофункционального инструмента: построение временных графиков по атрибутам MBean'ов, мониторинг heap/CPU/threads, снятие и анализ heap dump'ов (Classes, Instances, References, GC roots), CPU-сэмплирование с переключением между видами Hot Spots и Reverse Calls.
- Проведена самостоятельная локализация и устранение проблемы производительности в собственной программе с применением профилировщика, анализатора heap dump'а и инструментов браузера. Достигнуто ускорение стандартной пользовательской операции в ≈ 10 раз.

Основной вывод: мониторинг и оптимизация программы выполняются не с помощью одного инструмента, а путем комбинирования нескольких (Monitor, Sampler, Heap Dump, Network), что даёт полную картину узких мест программы и позволяет эффективно их устранить.